

Arrays and Strings

Overview

So far we have used only the fundamental data types, namely char, int, float, double and variations of int and double. Although these types are very useful, they are constrained by the fact that a variable of these types can store only one value at any given time. Therefore, they can be used only to handle limited amounts of data. In many applications, however, we need to handle a large volume of data in terms of reading, processing and printing. To process such large amounts of data, we need a powerful data type that would facilitate efficient storing, accessing and manipulation of data items. C supports a derived data type known as array that can be used for such applications.

What is an Array?

An array is a fixed-size sequenced collection of elements of the same data type. It is simply a grouping of like-type data. In its simplest form, an array can be used to represent a list of numbers, or a list of names. Some examples where the concept of an array can be used:

- List of temperatures recorded every hour in a day, or a month, or a year.
- List of employees in an organization.
- List of products and their cost sold by a store.
- Test scores of a class of students

and so on.

Since an array provides a convenient structure for representing data, it is classified as one of the data structures in C. Other data structures include structures, lists, queues and trees. A complete discussion of all data structures is beyond the scope of the syllabus, but we will also study structures in this unit ahead.

As we mentioned earlier, an array is a sequenced collection of related data items that share a common name. For instance, we can use an array name salary to represent a set of salaries of a group of employees in an organization. We can refer to the individual salaries by writing a number called index or subscript in brackets after the array name.

For example,

```
salary [10]
```

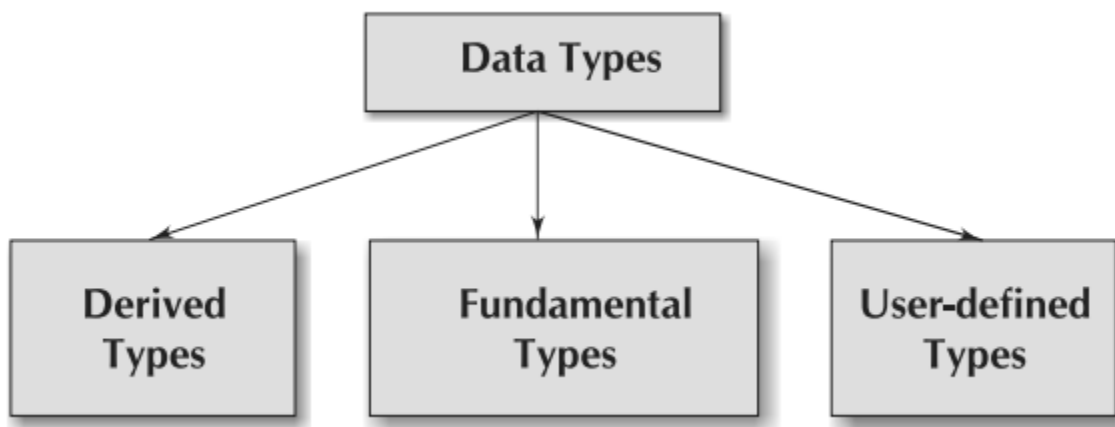
represents the salary of a 10th employee. While the complete set of values is referred to as an array, individual values are called elements. The ability to use a single name to represent a collection of items and to refer to an item by specifying the item number enables us to develop concise and efficient programs. For example, we can use a loop construct, discussed earlier, with the subscript as the control variable to read the entire array, perform calculations, and print out the results.

We can use arrays to represent not only simple lists of values but also tables of data in two, three or more dimensions. In this chapter, we introduce the concept of an array and discuss how to use it to create and apply the following types of arrays.

- One-dimensional arrays
- Two-dimensional arrays
- Multidimensional arrays

Data Structures

C supports a rich set of derived and user-defined data types in addition to a variety of fundamental types as shown below:



- Arrays
- Functions
- Pointers

- Integral Types
- Float Types
- Character Types

- Structures
- Unions
- Enumerations

Arrays and structures are referred to as structured data types because they can be used to represent data values that have a structure of some sort. Structured data types provide an organizational scheme that shows the relationships among the individual elements and facilitate efficient data manipulations. In programming parlance, such data types are known as data structures.

In addition to arrays and structures, C supports creation and manipulation of the following data structures:

- Linked Lists
- Stacks
- Queues
- Trees

One-Dimensional Arrays

A list of items can be given one variable name using only one subscript and such a variable is called a single-subscripted variable or a one-dimensional array.

In mathematics, we often deal with variables that are single-subscripted. For instance, we use the equation

$$A = \frac{\sum_{i=1}^n x_i}{n}$$

to calculate the average of n values of x. The subscripted variable x_i refers to the i th element of x. In C, a single-subscripted variable x_i can be expressed as $x[1]$, $x[2]$, $x[3]$,..... $x[n]$ The subscript can begin with number 0. That is $x[0]$ is allowed. Now we will discuss the properties of arrays and work with 1-D Arrays before we go on to discuss 2-D and multidimensional arrays.

Properties of Array

The array contains the following properties.

- Each element of an array is of the **same data type and carries the same size**, i.e., int = 4 bytes.
- Elements of the array are stored at **contiguous memory locations** where the first element is stored at the smallest memory location.
- Elements of the array can be **randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element**.

Declaration of Array

We can declare an array in the c language in the following way.

```
data_type array_name[array_size];
```

Now, let us see the example to declare the array.

```
int marks[5];
```

Here, int is the data_type, marks are the array_name, and 5 is the array_size.

Initialization of Array

The simplest way to initialize an array is by using the index of each element. We can initialize each element of the array by using the index. Consider the following example.

```
marks[0]=80;//initialization of array
marks[1]=60;
marks[2]=70;
marks[3]=85;
marks[4]=75;
```

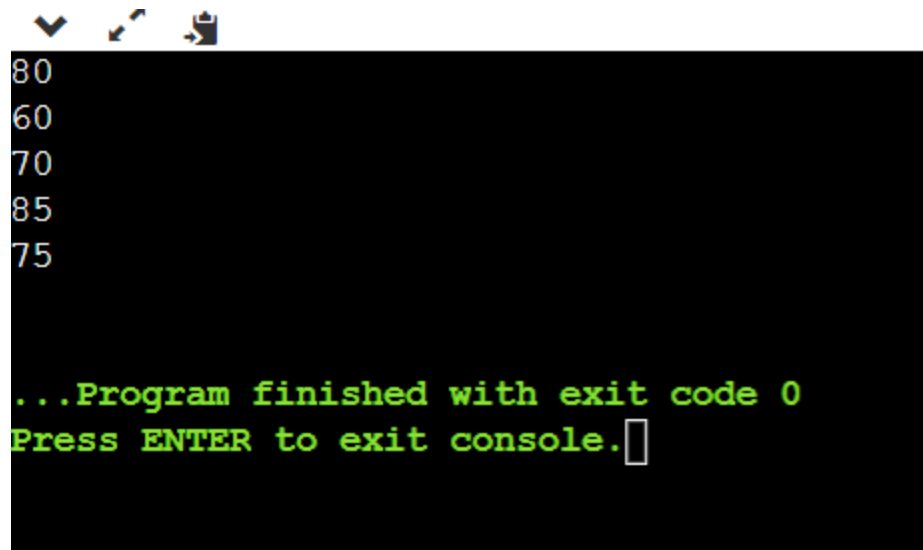
80	60	70	85	75
----	----	----	----	----

marks[0] marks[1] marks[2] marks[3] marks[4]

Initialization of Array

```
#include<stdio.h>
int main(){
int i=0;
int marks[5];//declaration of array
marks[0]=80;//initialization of array
marks[1]=60;
marks[2]=70;
marks[3]=85;
marks[4]=75;
//traversal of array
for(i=0;i<5;i++){
printf("%d \n",marks[i]);
} //end of for loop
return 0;
}
```

```
#include<stdio.h>
int main(){
int i=0;
int marks[5];//declaration of array
marks[0]=80;//initialization of array
marks[1]=60;
marks[2]=70;
marks[3]=85;
marks[4]=75;
//traversal of array
for(i=0;i<5;i++){
printf("%d \n",marks[i]);
} //end of for loop
return 0;
}
```

A terminal window with a black background and green text. At the top, there are three small icons: a checkmark, a cursor, and a clipboard. The text in the terminal shows the output of an array: 80, 60, 70, 85, 75. Below this, a message states: "...Program finished with exit code 0" and "Press ENTER to exit console." followed by a cursor icon.

```
80
60
70
85
75

...Program finished with exit code 0
Press ENTER to exit console.
```

Declaration with Initialization

We can initialize the c array at the time of declaration. Let's see the code.

```
int marks[5]={20,30,40,50,60};
```

In such a case, there is no requirement to define the size. So it may also be written as the following code.

```
int marks[]={20,30,40,50,60};
```

Let's see the C program to declare and initialize the array in C.

```
#include<stdio.h>
int main(){
int i=0;
int marks[5]={20,30,40,50,60}; //declaration and initialization of
array
//traversal of array
for(i=0;i<5;i++){
printf("%d \n",marks[i]);
}
return 0;
}
```

```
#include<stdio.h>
int main(){
int i=0;
int marks[5]={20,30,40,50,60};//declaration and initialization of array
//traversal of array
for(i=0;i<5;i++){
printf("%d \n",marks[i]);
}
return 0;
}
```

```
20
30
40
50
60
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Run Time Initialization

An array can be explicitly initialized at run time. This approach is usually applied for initializing large arrays. For example, consider the following segment of a C program.

```
- - - - -
- - - - -
for (i = 0; i < 100; i = i+1)
{
if i < 50
sum[i] = 0.0; /* assignment statement */
else
sum[i] = 1.0;
}
- - - - -
- - - - -
```

The first 50 elements of the array sum are initialized to zero while the remaining 50 elements are initialized to 1.0 at run time.

We can also use a read function such as scanf to initialize an array. For example, the statements

```
int x [3];
scanf("%d%d%d", &x[0], &x[1], &x[2]);
```

will initialize array elements with the values entered through the keyboard.

Some Code examples using 1D Arrays:

Program to Calculate Average Using Arrays:

```
#include <stdio.h>
int main() {
    int n, i;
    float num[100], sum = 0.0, avg;

    printf("Enter the numbers of elements: ");
    scanf("%d", &n);

    while (n > 100 || n < 1) {
        printf("Error! number should in range of (1 to 100).\n");
        printf("Enter the number again: ");
        scanf("%d", &n);
    }

    for (i = 0; i < n; ++i) {
        printf("%d. Enter number: ", i + 1);
        scanf("%f", &num[i]);
        sum += num[i];
    }

    avg = sum / n;
    printf("Average = %.2f", avg);
    return 0;
}
```

```

#include <stdio.h>
int main() {
    int n, i;
    float num[100], sum = 0.0, avg;

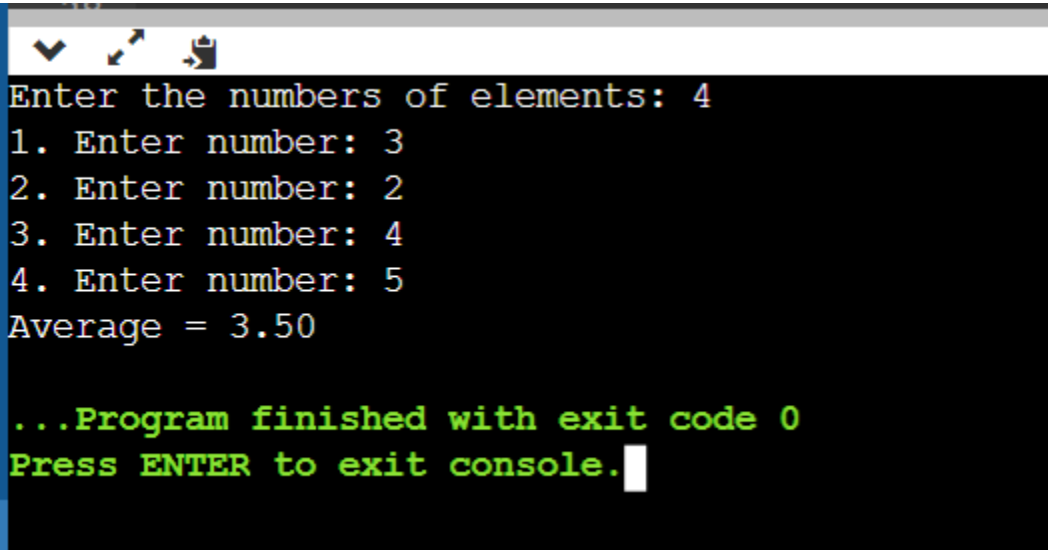
    printf("Enter the numbers of elements: ");
    scanf("%d", &n);

    while (n > 100 || n < 1) {
        printf("Error! number should in range of (1 to 100).\n");
        printf("Enter the number again: ");
        scanf("%d", &n);
    }

    for (i = 0; i < n; ++i) {
        printf("%d. Enter number: ", i + 1);
        scanf("%f", &num[i]);
        sum += num[i];
    }

    avg = sum / n;
    printf("Average = %.2f", avg);
    return 0;
}

```



```

Enter the numbers of elements: 4
1. Enter number: 3
2. Enter number: 2
3. Enter number: 4
4. Enter number: 5
Average = 3.50

...Program finished with exit code 0
Press ENTER to exit console.

```

Program to Find Largest Element in an Array:

```

#include <stdio.h>
int main() {

```



```

int n;
double arr[100];
printf("Enter the number of elements (1 to 100): ");
scanf("%d", &n);

for (int i = 0; i < n; ++i) {
    printf("Enter number%d: ", i + 1);
    scanf("%lf", &arr[i]);
}

// storing the largest number to arr[0]
for (int i = 1; i < n; ++i) {
    if (arr[0] < arr[i]) {
        arr[0] = arr[i];
    }
}

printf("Largest element = %.2lf", arr[0]);

return 0;
}

```

```

#include <stdio.h>
int main() {
    int n;
    double arr[100];
    printf("Enter the number of elements (1 to 100): ");
    scanf("%d", &n);

    for (int i = 0; i < n; ++i) {
        printf("Enter number%d: ", i + 1);
        scanf("%lf", &arr[i]);
    }

    // storing the largest number to arr[0]
    for (int i = 1; i < n; ++i) {
        if (arr[0] < arr[i]) {
            arr[0] = arr[i];
        }
    }

    printf("Largest element = %.2lf", arr[0]);

    return 0;
}

```

```
Enter the number of elements (1 to 100): 5
Enter number1: 4
Enter number2: 3
Enter number3: 1
Enter number4: 2
Enter number5: 5
Largest element = 5.00

...Program finished with exit code 0
Press ENTER to exit console.█
```

2D array

A two-dimensional array can be thought of as a rectangular display of elements with rows and columns. For example, elements of `int x[3][3]` are shown below:

Array Elements in Matrix Form

	Col1	Col2	Col3
Row1	<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>
Row2	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>
Row3	<code>x[2][0]</code>	<code>x[2][1]</code>	<code>x[2][2]</code>

Note: The arrangement of array elements shown here is only for the sake of understanding. Conceptually the elements are shown in a matrix form. Physically the array elements are stored in one continuous form in the memory.

A two-dimensional array is a collection of a number of one-dimensional arrays, which are placed one after another. For example, in the above table each row of a two-dimensional array can be thought of as a single dimensional array.

Initializing Two-Dimensional Arrays

Like the one-dimensional arrays, two-dimensional arrays may be initialized by following their declaration with a list of initial values enclosed in braces. For example,

```
int table[2][3] = { 0,0,0,1,1,1};
```

initializes the elements of the first row to zero and the second row to one. The initialization is done row by row. The above statement can be equivalently written as

```
int table[2][3] = {{0,0,0}, {1,1,1}};
```

by surrounding the elements of each row by braces.

We can also initialize a two-dimensional array in the form of a matrix as shown below:

```
int table[2][3] = {  
    {0,0,0},  
    {1,1,1}  
};
```

Note the syntax of the above statements. Commas are required after each brace that closes off a row, except in the case of the last row.

When the array is completely initialized with all values, explicitly, we need not specify the size of the first dimension. That is, the statement

```
int table [ ] [3] = {  
    { 0, 0, 0},  
    { 1, 1, 1}  
};
```

is permitted.

If the values are missing in an initializer, they are automatically set to zero. For instance, the statement:

```
int table[2][3] = {  
    {1,1},  
    {2}  
};
```

will initialize the first two elements of the first row to one, the first element of the second row to two, and all other elements to zero.

When all the elements are to be initialized to zero, the following short-cut method may be used.

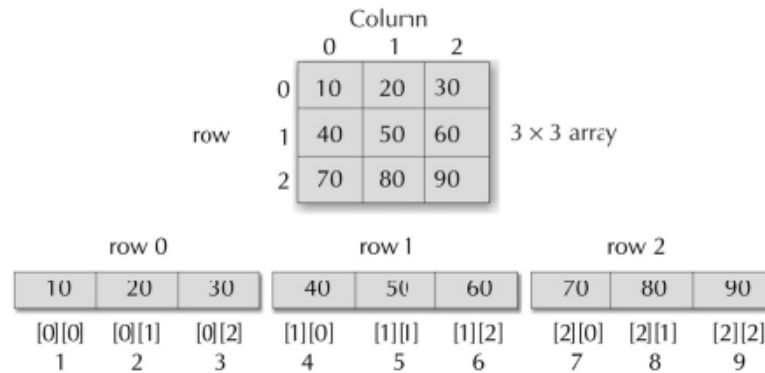
```
int m[3][5] = { {0}, {0}, {0}};
```

The first element of each row is explicitly initialized to zero while other elements are automatically initialized to zero. The following statement will also achieve the same result:

```
int m [3] [5] = { 0, 0};
```

Memory Layout

The subscripts in the definition of a two-dimensional array represent rows and columns. This format maps the way that data elements are laid out in the memory. The elements of all arrays are stored contiguously in increasing memory locations, essentially in a single list. If we consider the memory as a row of bytes, with the lowest address on the left and the highest address on the right, a simple array will be stored in memory with the first element at the left end and the last element at the right end. Similarly, a two-dimensional array is stored "row-wise", starting from the first row and ending with the last row, treating each row like a simple array. This is illustrated below.



Memory Layout

For a multi-dimensional array, the order of storage is that the first element stored has 0 in all its subscripts, the second has all of its subscripts 0 except the far right which has a value of 1 and so on.

The elements of a 2 x 3 x 3 array will be stored as under

1	2	3	4	5	6	7	8	9	
000	001	002	010	011	012	020	021	022	..
10	11	12	13	14	15	16	17	18	
..	100	101	102	110	111	112	120	121	122

The far right subscript increments first and the other subscripts increment in order from right to left. The sequence numbers 1, 2,, 18 represents the location of that element in the list

```
int main()
{
    int marks[5][2];
    int i,j;
    for(i=0;i<5;i++){
        for(j=0;j<2;j++){
            printf("\nEnter the value for [%d][%d]",i,j);
            scanf("%d",&marks[i][j]);
        }
    }
    for(i=0;i<5;i++){
        for(j=0;j<2;j++){
            printf("\t%d",marks[i][j]);

        }
        printf("\n");
    }
}
```

```
    return 0;
}
```

Program to find the largest and smallest numbers in a 2D Array

```
#include<stdio.h>
int main()
{
    int m, n, largest, smallest;
    int largrowloc, largcolumnloc, smallrowloc, smallcolumnloc;

    // take number of rows and columns
    printf("Enter number of row and column: ");
    scanf("%d %d",&m,&n);

    // declare array by given number of rows and columns
    int arr[m][n], i, j;

    // take array elements as input
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            printf("Enter arr[%d][%d]: ",i,j);
            scanf("%d", &arr[i][j]);
        }
        printf("\n");
    }

    // display array (optional)
    printf("Entered 2D Array:\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            printf("%d\t",arr[i][j]);
        }
        printf("\n");
    }

    // assume first element is
    // largest and smallest
    largest=arr[0][0];
    largrowloc=0;
```

```

largcolumnloc=0;
smallest=arr[0][0];
smallrowloc=0;
smallcolumnloc=0;

// compare with all elements
for(i=0;i<m;i++)
{
    for(j=0;j<n;j++)
    {
        if(largest<arr[i][j])
        {
            largest=arr[i][j];
            largrowloc=i; //row location of largest element
            largcolumnloc=j; //column location of largest element
        }

        if(smallest>arr[i][j])
        {
            smallest=arr[i][j];
            smallrowloc=i; //row location of smallest element
            smallcolumnloc=j; //column location of smallest element
        }
    }
}

// display results
printf("\n"); // new line
printf("Largest element in array is %d in location arr[%d][%d]\n",
       largest, largrowloc, largcolumnloc);
printf("Smallest element in array is %d in location arr[%d][%d]\n",
       smallest, smallrowloc, smallcolumnloc);

return 0;
}

```

```
Enter arr[1][1]: 4

Entered 2D Array:
1      2
3      4

Largest element in array is 4 in location arr[1][1]
Smallest element in array is 1 in location arr[0][0]
```

Program to check whether a 3x3 Matrix is symmetric matrix

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int A[3][3]; // Original matrix
```

```
    int B[3][3]; // Transpose matrix
```

```
    int row, col, isSymmetric;
```

```
    /* Input elements in matrix A from user */
```

```
    printf("Enter elements in matrix of size 3x3: \n");
```

```
    for(row=0; row<3; row++)
```

```
    {
```

```
        for(col=0; col<3; col++)
```

```
        {
```

```
            scanf("%d", &A[row][col]);
```

```
        }
```

```
    }
```

```
    /*
```

```
    * Find transpose of matrix A
```

```
    */
```

```
    for(row=0; row<3; row++)
```

```
    {
```

```
        for(col=0; col<3; col++)
```

```
        {
```

```
            /* Store each row of matrix A to each column of matrix B */
```

```
            B[row][col] = A[col][row];
```

```
        }
```

```
    }
```

```
    /*
```

```

    * Check whether matrix A is equal to its transpose or not
    */
    isSymmetric = 1;
    for(row=0; row<3 && isSymmetric; row++)
    {
        for(col=0; col<3; col++)
        {
            /* If matrix A is not equal to its transpose */
            if(A[row][col] != B[row][col])
            {
                isSymmetric = 0;
                break;
            }
        }
    }

    /*
    * If the given matrix is symmetric.
    */
    if(isSymmetric == 1)
    {
        printf("\nThe given matrix is Symmetric matrix: \n");

        for(row=0; row<3; row++)
        {
            for(col=0; col<3; col++)
            {
                printf("%d ", A[row][col]);
            }

            printf("\n");
        }
    }
    else
    {
        printf("\nThe given matrix is not Symmetric matrix.");
    }

    return 0;
}

```



```
Enter elements in matrix of size 3x3:
1
2
3
2
2
5
3
5
3

The given matrix is Symmetric matrix:
1 2 3
2 2 5
3 5 3

...Program finished with exit code 0
Press ENTER to exit console.
```

Strings and Character Array

String is a sequence of characters that are treated as a single data item and terminated by a null character '\0'. Remember that the C language does not support strings as a data type. A string is actually a one-dimensional array of characters in C language. These are often used to create meaningful and readable programs.

Declaration of strings:

Declaring a string is as simple as declaring a one-dimensional array. Below is the basic syntax for declaring a string.

```
char str_name[size];
```

In the above syntax str_name is any name given to the string variable and size is used to define the length of the string, i.e the number of characters strings will store. Please keep in mind that there is an extra terminating character which is the **Null character ('\0')** used to indicate the termination of string which differs strings from normal character arrays.

Initializing a String:

A string can be initialized in different ways. We will explain this with the help of an example. Below is an example to declare a string with name as str and initialize it with "c string"

1. `char str[] = "c string";`
2. `char str[50] = "c string";`
3. `char str[] = {'c',' ','s','t','r','i','n','g','\0'};`
4. `char str[14] = {'c',' ','s','t','r','i','n','g','\0'};`

When the compiler encounters a sequence of characters enclosed in the double quotation marks, it appends a null character `\0` at the end by default.

c		s	t	r	i	n	g	\0
---	--	---	---	---	---	---	---	----

What if the length of the string is more than the size of the character array?

```
#include <stdio.h>
int main()
{
    char ch[3] = "hello";
    printf(" %s", ch);
    return 0;
}
```

```
main.c:12:18: warning: initializer-string for array of chars is too long
 12 |     char ch[3] = "hello";
    |                  ^~~~~~
hel

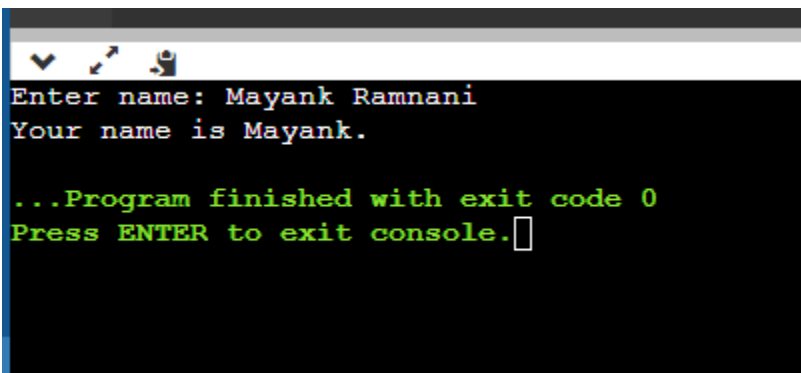
...Program finished with exit code 0
Press ENTER to exit console.
```

Read String from the user

You can use the `scanf()` function to read a string.

The `scanf()` function reads the sequence of characters until it encounters whitespace (space, newline, tab, etc.).

```
#include <stdio.h>
int main()
{
    char name[20];
    printf("Enter name: ");
    scanf("%s", name);
    printf("Your name is %s.", name);
    return 0;
}
```



```
Enter name: Mayank Ramnani
Your name is Mayank.

...Program finished with exit code 0
Press ENTER to exit console.
```

notice that we have used the code name instead of &name with scanf().

```
scanf("%s", name);
```

This is because the name is a char array, and we know that array names decay to pointers in C.

Thus, the name in scanf() already points to the address of the first element in the string, which is why we don't need to use &.

String Input and Output:

- %s format specifier to read a string input from the terminal.
- But scanf() function terminates its input on the first white space it encounters.
- The fgets() function can also be used to read character string with white spaces

String Handling Functions:

C language supports a large number of string handling functions that can be used to carry out many of the string manipulations. These functions are packaged in the string.h library. Hence, you must include the **string.h** header file in your programs to use these functions.

The following are the most commonly used string handling functions.

Method	Description
strcat()	It is used to concatenate(combine) two strings
strlen()	It is used to show the length of a string
strrev()	It is used to show the reverse of a string
strcpy()	Copies one string into another
strcmp()	It is used to compare two string

How to read a line of text?

You can use the gets() and fgets() function to read a line of string. And, you can use puts() to display the string.

```

#include <stdio.h>

int main()
{

char str[20];
gets(str);
printf("%s", str);


    return 0;
}

```

input

```

In file included from main.c:9:
/usr/include/stdio.h:577:14: note: declared here
 577 | extern char *gets (char *__s) __wur __attribute_deprecated__;
      | ^~~~~
/usr/bin/ld: /tmp/ccMUTPIV.o: in function `main':
main.c:(.text+0x23): warning: the `gets' function is dangerous and should not be used.
mayank
mayank

...Program finished with exit code 0
Press ENTER to exit console.

```

```

9  #include <stdio.h>
10
11 int main(void)
12 {
13     char name[20];
14
15     printf("Enter your name: ");
16     scanf("%s", name);
17     // gets(name);
18     printf("Your Name is: %s", name);
19
20     return 0;
21 }

```

```

Enter your name: Mayank Ramnani
Your Name is: Mayank

...Program finished with exit code 0
Press ENTER to exit console.

```

```
9  #include <stdio.h>
10
11 int main(void)
12 {
13     char name[20];
14
15     printf("Enter your name: ");
16     // scanf("%s", name);
17     gets(name);
18     printf("Your Name is: %s", name);
19
20     return 0;
21 }
```

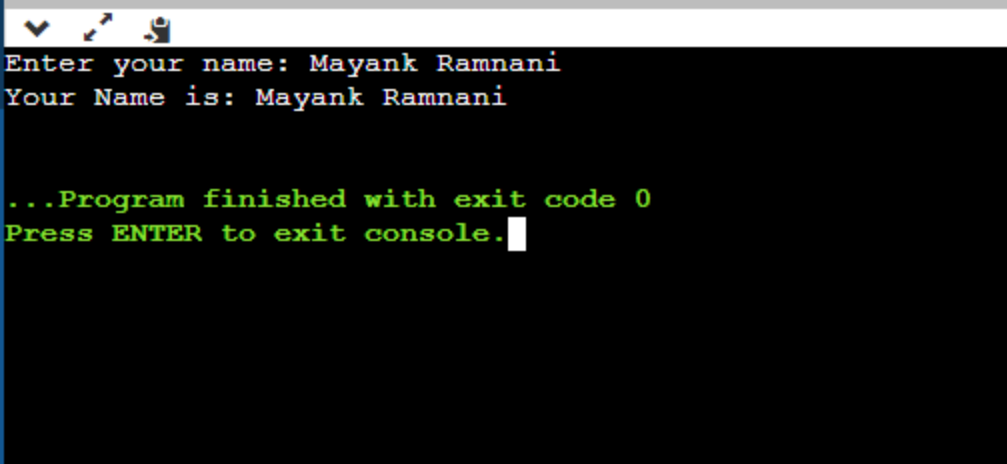
input

```
main.c:17:4: warning: 'gets' is deprecated [-Wdeprecated-declarations]
   17 |     gets(name);
      |     ^~~~
In file included from main.c:9:
/usr/include/stdio.h:577:14: note: declared here
   577 | extern char *gets (char *__s) __wur __attribute_deprecated__;
      |              ^~~~
/usr/bin/ld: /tmp/cc23C2Xd.o: in function `main':
main.c:(.text+0x34): warning: the `gets' function is dangerous and should not be used.
Enter your name: Mayank Ramnani
Your Name is: Mayank Ramnani

...Program finished with exit code 0
Press ENTER to exit console.
```

But gets() is deprecated, we can also use **fgets()**

```
8
9  #include <stdio.h>
10
11  int main(void)
12  {
13      char name[20];
14
15      printf("Enter your name: ");
16      // scanf("%s", name);
17      // gets(name);
18      fgets (name, 20, stdin);
19      printf("Your Name is: %s", name);
20
21      return 0;
22  }
```



```
Enter your name: Mayank Ramnani
Your Name is: Mayank Ramnani

...Program finished with exit code 0
Press ENTER to exit console.
```

fgets() vs gets(): fgets() is safer

fgets() reads a line from the specified stream and stores it into the string pointed to by str. It stops when either (n-1) characters are read, the newline character is read, or the end-of-file is reached, whichever comes first.

Syntax :

```
char *fgets(char *str, int n, FILE *stream)
```

str : Pointer to an array of chars where the string read is copied.

n : Maximum number of characters to be copied into str
(including the terminating null-character).

*stream : Pointer to a FILE object that identifies an input stream.
stdin can be used as argument to read from the standard input.

returns : the function returns str

It is safe to use because it checks the array bound.

It keeps on reading until a new line character is encountered or the maximum limit of the character array.

Suppose we have a character array of 15 characters and input is greater than 15 characters, gets() will read all these characters and store them into variable. Since, gets() do not check the maximum limit of input characters, so at any time compiler may return buffer overflow error.

```
#include <stdio.h>

int main(void)
{
    char str[15];

    printf("Enter string: ");
    gets(str);
    printf("Your string is: %s", str);

    return 0;
}
```



```

8
9  #include <stdio.h>
10
11 int main(void)
12 {
13     char str[15];
14
15     printf("Enter string: ");
16     gets(str);
17     printf("Your string is: %s", str);
18
19     return 0;
20 }

```

input

```

main.c:16:5: warning: 'gets' is deprecated [-Wdeprecated-declarations]
   16 |     gets(str);
      |     ^~~~
In file included from main.c:9:
/usr/include/stdio.h:577:14: note: declared here
   577 | extern char *gets (char *__s) __wur __attribute_deprecated__;
      |           ^~~~
/usr/bin/ld: /tmp/ccXaHreV.o: in function `main':
main.c:(.text+0x34): warning: the `gets' function is dangerous and should not be used.
Enter string: Hello and its very nice to meet you, I hope you are fine.
*** stack smashing detected ***: terminated

...Program finished with exit code 0
Press ENTER to exit console.

```

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      char str[15];
6
7      printf("Enter string: ");
8      gets(str);
9      printf("Your string is: %s", str);
10
11     return 0;
12 }
13
```

"C:\Users\Mayank\Desktop\C Programs\getsEx.exe"

Enter string: Hello and very nice to meet you , hope you are fine.
Your string is: Hello and very nice to meet you , hope you are fine.
Process returned -1073741819 (0xC0000005) execution time : 24.290 s
Press any key to continue.

"C:\Users\Mayank\Desktop\C Programs\getsEx.exe"

Enter string: Hello and very nice to meet you , hope you are fine.
Your string is: Hello and very nice to meet you , hope you are fine.
Process returned -1073741819 (0xC0000005) execution time : 24.290 s
Press any key to continue.

```
mayank@DESKTOP-OD92JV9: ~
GNU nano 2.9.3 getsExample.c

#include <stdio.h>

int main(void)
{
    char str[15];

    printf("Enter string: ");
    gets(str);
    printf("Your string is: %s", str);

    return 0;
}

[ Read 14 lines ]
^G Get Help      ^O Write Out     ^W Where Is      ^K Cut Text      ^J Justify       ^C Cut
^X Exit          ^R Read File     ^\ Replace       ^U Uncut Text    ^T To Spell      ^_ Go
```

```
mayank@DESKTOP-OD92JV9: ~
mayank@DESKTOP-OD92JV9:~$ gcc getsExample.c
getsExample.c: In function 'main':
getsExample.c:8:5: warning: implicit declaration of function 'gets'; did you mean 'fgets'? [-Wimplicit-function-declaration]
    gets(str);
    ~~~~~
    fgets
/tmp/ccazuWml.o: In function `main':
getsExample.c:(.text+0x35): warning: the `gets' function is dangerous and should not be used.
mayank@DESKTOP-OD92JV9:~$ ./a.out
Enter string: Hello and welcome to c programming
*** stack smashing detected ***: <unknown> terminated
Aborted (core dumped)
mayank@DESKTOP-OD92JV9:~$
```

That is why gets() is dangerous to use, it does not check the array bounds, while fgets() is safer, because it checks the array bound. It keeps on reading until a new line character is encountered or the maximum limit of the character array.

```
8  *****
9  #include <stdio.h>
10
11 int main(void)
12 {
13     char str[15];
14
15     printf("Enter string: ");
16     fgets(str, 15, stdin);
17     printf("Your string is: %s", str);
18
19     return 0;
20 }
```

Enter string: Hello and welcome to C programming.
Your string is: Hello and welc

...Program finished with exit code 0
Press ENTER to exit console.

```

8
9  #include <stdio.h>
10
11 int main(void)
12 {
13     char str[15];
14
15     printf("Enter string: ");
16     fgets(str, 15, stdin);
17     printf("Your string is: %s", str);
18
19     return 0;
20 }

```



Enter string: Hello

Your string is: Hello

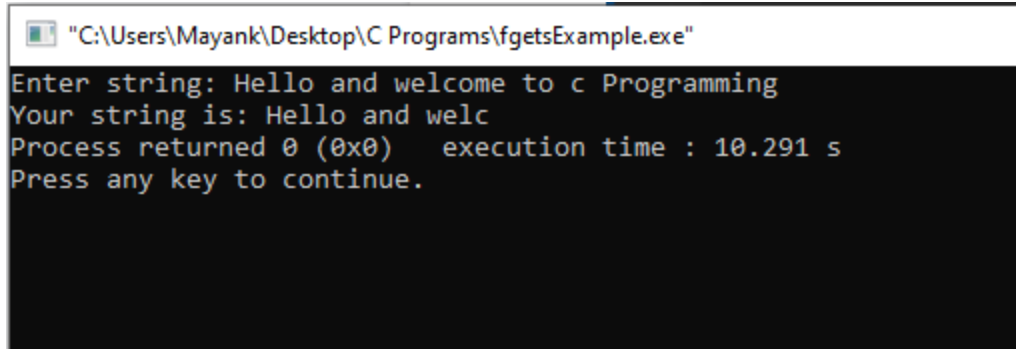
...Program finished with exit code 0

Press ENTER to exit console.

```

1
2  #include <stdio.h>
3
4  int main(void)
5  {
6      char str[15];
7
8      printf("Enter string: ");
9      fgets(str, 15, stdin);
10     printf("Your string is: %s", str);
11
12     return 0;
13 }
14

```



```
"C:\Users\Mayank\Desktop\C Programs\fgetsExample.exe"
Enter string: Hello and welcome to c Programming
Your string is: Hello and welc
Process returned 0 (0x0) execution time : 10.291 s
Press any key to continue.
```

Display Of Strings With Different Formats

The printf() function is used for displaying the various data types. The printf() function with %s format specifier is to be used for displaying the string on the screen. Various formats are shown in the code below:

```
#include <stdio.h>

int main()
{
    char text[15]= "Hello World";
    printf("%s\n",text);
    printf("%.5s\n",text);
    printf("%.8s\n",text);
    printf("%.11s\n",text);
    printf("%-10.4s\n",text);
    return 0;
}
```

```
9  #include <stdio.h>
10
11  int main()
12  {
13      char text[15]= "Hello World";
14      printf("%s\n",text);
15      printf("%.5s\n",text);
16      printf("%.8s\n",text);
17      printf("%.11s\n",text);
18      printf("%-10.4s\n",text);
19      return 0;
20  }
21
```

input

Hello World
Hello
Hello Wo
Hello World
Hell
Hello World

...Program finished with exit code 0
Press ENTER to exit console.

Let us understand the statements we used in the code , numbered below:

1. `printf("%s\n",text);`
2. `printf("%.5s\n",text);`
3. `printf("%.8s\n",text);`
4. `printf("%.11s\n",text);`
5. `printf("%-10.4s\n",text);`

1. The 1st statement displays the output 'Hello World'. The entire string can be displayed with the first statement.
2. We can also specify the precision with a format string. The precision (number of characters to be displayed) is provided after the decimal point. Consider the 2nd statement in the above table (the first five characters are displayed). Here the integer value 5 on the right side of the decimal point specifies the five characters to be displayed.
3. In the 3rd statement, the first eight characters are displayed, while statement 4 displays the entire string.
4. The 5th statement with—sign (`% - 10.4s`), displays the string with left justified.

When the field length is less than the length of the string, the entire string is printed. When it is greater than the length of the string, blank spaces are initially printed followed by the string.

Additionally, when the number of characters to be displayed is specified as zero after the decimal point, nothing will be displayed.

Another Example:

```
#include <stdio.h>

int main()
{
    char str[15]="Hello World";
    printf("%-16.2s\n",str);
    printf("%17s\n",str);
    printf("%18s\n",str);
    printf("%19s\n",str);
    printf("%18s\n",str);
    printf("%17s\n",str);
    printf("%16s\n",str);
    printf("%s\n",str);
    for(int i=0; i<10;i++){
        for(int j=0;j<5;j++){
            // printf("%s\t",str);
            printf("%17s",str);
        }
        printf("\n");
    }
    return 0;
}
```



```

He
    Hello World
      Hello World
        Hello World
          Hello World
            Hello World
              Hello World
Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
Hello World   Hello World   Hello World   Hello World   Hello World
...Program finished with exit code 0
Press ENTER to exit console.
```

Puts function in C

The puts function in C is used to write a line or string to the output stream (stdout) that is up to, but does not include, the null character. The puts function also appends a newline character to the output and returns an integer.

The prototype of the puts function is shown below:

```
int puts(const char* str);
```

The puts function takes a single mandatory parameter, i.e., a null-terminated character array. The puts function writes the provided argument to the output stream and appends a newline character at the end. If the execution is successful, the function returns a non-negative integer; otherwise, it returns an EOF (End-of-File) for any error.

```
#include <stdio.h>
#include <string.h>

int main() {

    // initializing strings
    char str1[] = "Hello World";
    char str2[] = "Using puts in C";
```

```

    // writing to stdout
    puts(str1);
    puts(str2);

    return 0;
}

9  #include <stdio.h>
10 #include <string.h>
11
12 int main() {
13
14     // initializing strings
15     char str1[] = "Hello World";
16     char str2[] = "Using puts in C";
17
18     // writing to stdout
19     puts(str1);
20     puts(str2);
21
22     return 0;
23 }
```



```

Hello World
Using puts in C
```

```

...Program finished with exit code 0
Press ENTER to exit console.
```

String Functions:

strlen(): The strlen() function returns the length of the given string. It doesn't count null character '\0'.

```

#include <stdio.h>
#include <string.h>
int main()
{
    char text[15]= "Hello World";
    char ch='*'; // initialized to a random character to avoid any problems due to garbage value
    int ctr=0;
    for(int i=1;ch!='\0';i++){
        ch=text[i];
        ctr++;
    }
    printf("lenght is %u",ctr);
    printf("\nlenght is %u",strlen(text));
    return 0;
}

```

input

```

main.c:13:23: warning: format '%u' expects argument of type 'unsigned int', but argument 2 has type 'size_t' {aka 'long unsigned int'} [-Wformat=]
   13 | printf("\nlenght is %u",strlen(text));
      |                ^~ ~~~~~
      |                | |
      |                | size_t {aka long unsigned int}
      |                unsigned int
      |                %lu
lenght is 11
lenght is 11

```

strcpy(): The strcpy(destination, source) function copies the source string in destination.

```

1  #include<stdio.h>
2  #include <string.h>
3  int main(){
4  char ch[20]="Hello World" ;
5  char ch2[20];
6  strcpy(ch2,ch);
7  printf("Value of second string is: %s",ch2);
8  return 0;
9  }
10

```

```

Value of second string is: Hello World

...Program finished with exit code 0
Press ENTER to exit console.

```

strcat(): The `strcat(first_string, second_string)` function concatenates two strings and the result is returned to `first_string`.

```
main.c
1  #include<stdio.h>
2  #include <string.h>
3  int main(){
4      char ch[10]="Hello";
5      char ch2[10]={'C', '\0'};
6      strcat(ch,ch2);
7      printf("Value of first string is: %s",ch);
8      printf("\nValue of second string is: %s",ch2);
9      return 0;
10 }
11
```

Value of first string is: HelloC
Value of second string is: C

...Program finished with exit code 0
Press ENTER to exit console.

We must ensure that the size of the `first_string` (to which the second string is appended) is large enough to accommodate the final string (to avoid absurd results).

```
main.c
1  #include<stdio.h>
2  #include <string.h>
3  int main(){
4      char ch[10]="Hello";
5      char ch2[10]={'program', '\0'};
6      strcat(ch,ch2);
7      printf("Value of first string is: %s",ch);
8      printf("\nValue of second string is: %s",ch2);
9      return 0;
10 }

input
main.c:5:18: warning: character constant too long for its type
5 |   char ch2[10]={'program', '\0'};
  |                   ^~~~~~
main.c:5:18: warning: overflow in conversion from 'int' to 'char' changes value from '1735549293' to '109' [-Woverflow]
Value of first string is: Hellom
Value of second string is: m

...Program finished with exit code 0
Press ENTER to exit console.
```

strcmp(): The strcmp(first_string, second_string) function compares two string and returns 0 if both strings are equal.

```
1  #include<stdio.h>
2  #include <string.h>
3  int main(){
4      char str1[20],str2[20];
5      printf("Enter 1st string: ");
6      fgets(str1,20,stdin);
7      printf("Enter 2nd string: ");
8      fgets(str2,20,stdin);
9      if(strcmp(str1,str2)==0)
10         printf("Strings are equal");
11     else
12         printf("Strings are not equal");
13     return 0;
14 }
```

```
Enter 1st string: Mayank
Enter 2nd string: Mayank
Strings are equal

...Program finished with exit code 0
Press ENTER to exit console.
```

strrev(): The strrev(string) function returns the reverse of the given string. Let's see a simple example of strrev() function.

```
Start here x strreverse.c x
1  #include<stdio.h>
2  #include<string.h>
3
4  int main()
5  {
6  char str[20] = "Hello World";
7
8  printf("The given string is =%s\n",str);
9
10 printf("After reversing string is =%s",strrev(str));
11
12 return 0;
13 }
14
```

```
"D:\Work\SGSITS\BE First year C\strreverse.exe"
The given string is =Hello World
After reversing string is =dlrow olleH
Process returned 0 (0x0)   execution time : 5.968 s
Press any key to continue.
```